

INSTITUTO FEDERAL DE EDUCAÇÃO CIÊNCIA E TECNOLOGIA
DE MINAS GERAIS - *CAMPUS* SÃO JOÃO EVANGELISTA
BACHARELADO EM SISTEMAS DE INFORMAÇÃO

Valdir de Souza Carvalho Neto

Atividade 05 - Atividade sobre o sistema operacional Linux Mint

Sistemas Operacionais

São João Evangelista

2026

LISTA DE ILUSTRAÇÕES

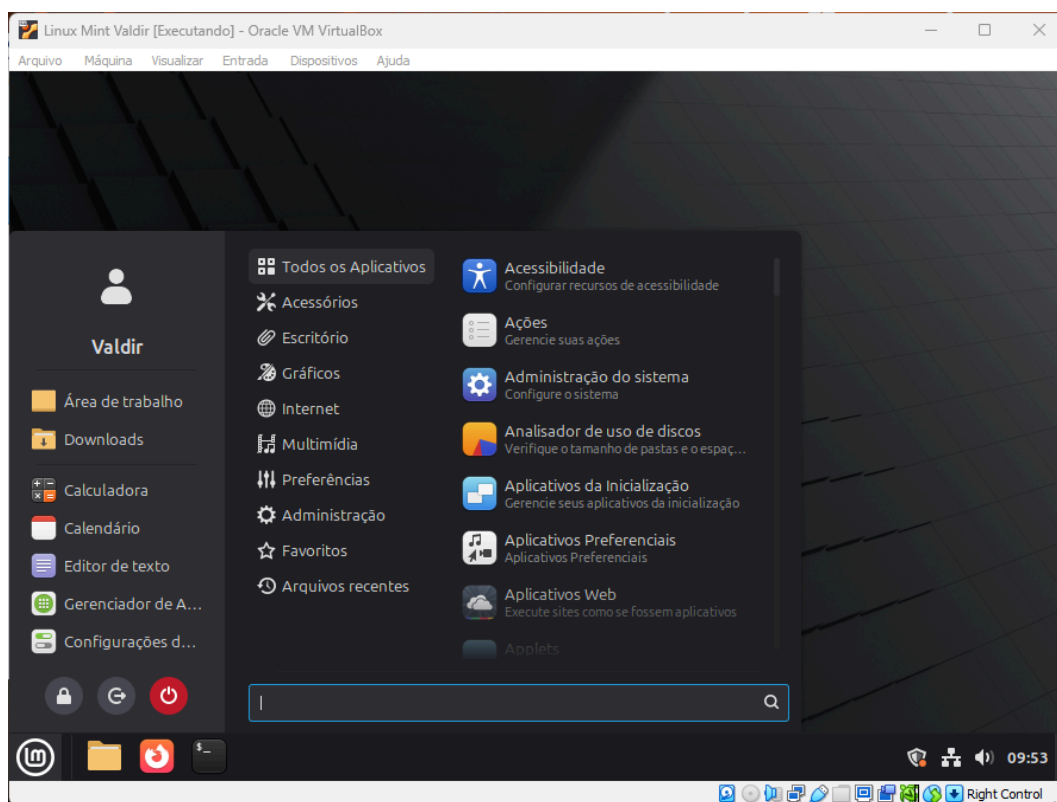
Figura 1. Sistema Linux instalado.....	4
Figura 2. Captura de tela do comando (sudo).....	5
Figura 3. Captura de tela do comando (sudo su).....	5
Figura 4. Captura de tela do comando (ls).....	6
Figura 5. Captura de tela do comando (ls -l).....	6
Figura 6. Captura de tela do comando (ls -a).....	7
Figura 7. Captura de tela do comando (cd).....	7
Figura 8. Captura de tela do comando (pwd).....	7
Figura 9. Captura de tela do comando (mkdir).....	8
Figura 10. Captura de tela do comando (rmdir).....	8
Figura 11. Captura de tela do comando (touch).....	9
Figura 12. Captura de tela do comando (nano).....	9
Figura 13. Captura de tela do comando (cp).....	10
Figura 14. Captura de tela do comando (mv).....	10
Figura 15. Captura de tela do comando (rm).....	10
Figura 16. Captura de tela do comando (cat).....	11
Figura 17. Captura de tela do comando (ps).....	11
Figura 18. Captura de tela do comando (top).....	12
Figura 19. Captura de tela do comando (htop).....	13
Figura 20. Captura de tela do comando (htop).....	13
Figura 21. Captura de tela do comando (kill).....	14
Figura 22. Captura de tela do comando (killall).....	15
Figura 23. Captura de tela do comando (jobs).....	16
Figura 24. Captura de tela do comando (bg).....	16
Figura 25. Captura de tela do comando (fg).....	17
Figura 26. Captura de tela do comando (uname -a).....	17
Figura 27. Captura de tela do comando (uname -r).....	17
Figura 28. Captura de tela do comando (hostname).....	18
Figura 29. Captura de tela do comando (df -h).....	18
Figura 30. Captura de tela do comando (du -sh).....	19
Figura 31. Captura de tela do comando (lscpu).....	20
Figura 32. Captura de tela do comando (who).....	21
Figura 33. Captura de tela do comando (whoami).....	21

SUMÁRIO

1. INTRODUÇÃO.....	4
2. DESENVOLVIMENTO.....	5
2.1. Comando sudo.....	5
2.2. Comando sudo su.....	5
2.3. Comando ls.....	5
2.4. Comando ls -l.....	6
2.5. Comando ls -a.....	6
2.6. Comando cd.....	7
2.7. Comando pwd.....	7
2.8. Comando mkdir.....	8
2.9. Comando rmdir.....	8
2.10. Comando touch.....	8
2.11. Comando nano.....	9
2.12. Comando cp.....	9
2.13. Comando mv.....	10
2.14. Comando rm.....	10
2.15. Comando cat.....	11
2.16. Comando ps.....	11
2.17. Comando top.....	11
2.18. Comando htop.....	12
2.19. Comando kill.....	14
2.20. Comando killall.....	14
2.21. Comando jobs.....	15
2.22. Comando bg.....	16
2.23. Comando fg.....	16
2.24. Comando uname -a.....	17
2.25. Comando uname -r.....	17
2.26. Comando hostname.....	18
2.27. Comando df -h.....	18
2.28. Comando du -sh.....	18
2.29. Comando lscpu.....	19
2.30. Comando who.....	20
2.31. Comando whoami.....	21
3. CONSIDERAÇÕES FINAIS.....	22
3.1. Comandos mais úteis.....	22
3.2. Comandos com mais dificuldade.....	22
3.3. Como o Shell se relaciona com os conceitos de Sistemas Operacionais estudados em sala.....	22

1. INTRODUÇÃO

Figura 1. Sistema Linux instalado



Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

O Linux Mint é um sistema operacional moderno, classificado como multiprogramável e multitarefa, que atua como o principal gerenciador de recursos (*hardware* e *software*) do computador. Para interagir com o núcleo desse sistema (o Kernel), utilizamos interfaces. O Shell é exatamente isso: uma interface de linha de comando que opera no "Modo Usuário". Ele atua como um tradutor, recebendo as instruções digitadas em texto, interpretando-as e enviando as solicitações para que o Kernel execute as operações diretamente no *hardware*.

2. DESENVOLVIMENTO

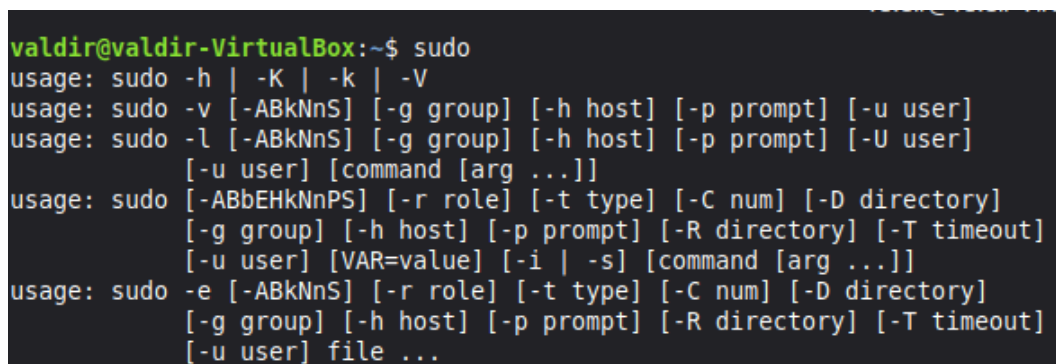
2.1. Comando *sudo*

Finalidade: Permite executar um comando com os privilégios de segurança do superusuário (root/administrador). É usado quando um comando exige permissões elevadas.

Sintaxe básica: `sudo [comando]`

Resultado obtido: O terminal exibiu uma mensagem dizendo todos os comandos possíveis para o `sudo`.

Figura 2. Captura de tela do comando (`sudo`)



```
valdir@valdir-VirtualBox:~$ sudo
usage: sudo -h | -K | -k | -V
usage: sudo -v [-ABkNnS] [-g group] [-h host] [-p prompt] [-u user]
usage: sudo -l [-ABkNnS] [-g group] [-h host] [-p prompt] [-U user]
           [-u user] [command [arg ...]]
usage: sudo [-ABbEHkNnPS] [-r role] [-t type] [-C num] [-D directory]
           [-g group] [-h host] [-p prompt] [-R directory] [-T timeout]
           [-u user] [VAR=value] [-i | -s] [command [arg ...]]
usage: sudo -e [-ABkNnS] [-r role] [-t type] [-C num] [-D directory]
           [-g group] [-h host] [-p prompt] [-R directory] [-T timeout]
           [-u user] file ...
```

Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

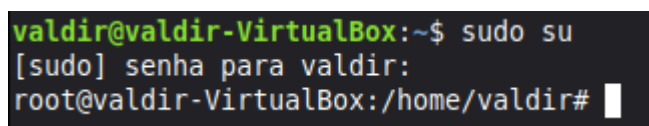
2.2. Comando *sudo su*

Finalidade: Alterna a sua sessão atual permanentemente para o usuário root (superusuário). Assim, o usuário não precisa digitar `sudo` antes de cada comando subsequente.

Sintaxe básica: `sudo su`

Resultado obtido: O prompt de comando mudou, terminando com `#` em vez de `$`, indicando que a sessão atual passou a operar continuamente como o superusuário (root).

Figura 3. Captura de tela do comando (`sudo su`)



```
valdir@valdir-VirtualBox:~$ sudo su
[sudo] senha para valdir:
root@valdir-VirtualBox:/home/valdir#
```

Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

2.3. Comando *ls*

Finalidade: Lista os arquivos e diretórios presentes dentro do diretório atual em que o usuário está navegando.

Sintaxe básica: `ls` ou `ls [caminho_do_diretorio]`

Resultado obtido: O terminal exibiu uma lista simples com os nomes das pastas e arquivos visíveis presentes no diretório atual, geralmente organizados em colunas e diferenciados por cores.

Figura 4. Captura de tela do comando (`ls`)

```
valdir@valdir-VirtualBox:~$ ls
'Área de trabalho'  Documentos  Downloads  Imagens  Modelos  Músicas  Público  Vídeos
```

Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

2.4. Comando `ls -l`

Finalidade: Lista o conteúdo do diretório em formato longo (detalhado), mostrando permissões, dono do arquivo, tamanho e data da última modificação.

Sintaxe básica: `ls -l` ou `ls -l [caminho]`

Resultado obtido: Foi exibida uma lista em formato de texto detalhado (uma linha por arquivo), mostrando as permissões (ex: `drwxr-xr-x`), número de links, proprietário, tamanho em bytes e a data de modificação de cada item.

Figura 5. Captura de tela do comando (`ls -l`)

```
valdir@valdir-VirtualBox:~$ ls -l
total 32
drwxr-xr-x 2 valdir valdir 4096 mai 11 09:51 'Área de trabalho'
drwxr-xr-x 2 valdir valdir 4096 mai 11 09:51 Documentos
drwxr-xr-x 2 valdir valdir 4096 mai 11 09:51 Downloads
drwxr-xr-x 2 valdir valdir 4096 mai 11 09:51 Imagens
drwxr-xr-x 2 valdir valdir 4096 mai 11 09:51 Modelos
drwxr-xr-x 2 valdir valdir 4096 mai 11 09:51 Músicas
drwxr-xr-x 2 valdir valdir 4096 mai 11 09:51 Público
drwxr-xr-x 2 valdir valdir 4096 mai 11 09:51 Vídeos
```

Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

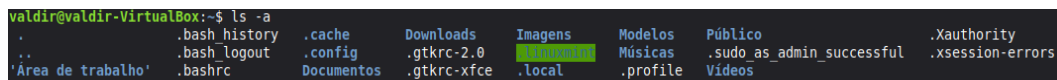
2.5. Comando `ls -a`

Finalidade: Lista todos os arquivos do diretório, incluindo os arquivos e pastas ocultos (que começam com um ponto “.”).

Sintaxe básica: `ls -a`

Resultado obtido: Ao executar o comando no diretório atual, o terminal exibiu a lista de pastas normais e também revelou arquivos de configuração ocultos, como o `.bashrc` e `.profile`.

Figura 6. Captura de tela do comando (`ls -a`)



```
valdir@valdir-VirtualBox:~$ ls -a
.      .bash_history  .cache      Downloads  Imagens     Modelos     Público     .Xauthority
..     .bash_logout  .config     .gtkrc-2.0  .local      Músicas     .sudo_as_admin_successful .xsession-errors
'Área de trabalho' .bashrc       Documentos  .gtkrc-xfce .profile    Vídeos
```

Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

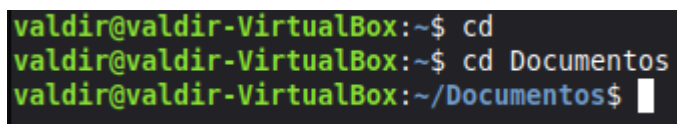
2.6. Comando `cd`

Finalidade: *Change Directory*. Usado para navegar entre as pastas do sistema operacional.

Sintaxe básica: `cd [caminho_do_diretorio]`

Resultado obtido: O comando foi executado silenciosamente. A única mudança visível foi o indicador de diretório no próprio prompt de comando, que foi atualizado para refletir o novo local acessado.

Figura 7. Captura de tela do comando (`cd`)



```
valdir@valdir-VirtualBox:~$ cd
valdir@valdir-VirtualBox:~$ cd Documentos
valdir@valdir-VirtualBox:~/Documentos$
```

Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

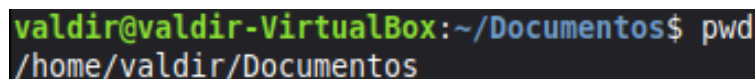
2.7. Comando `pwd`

Finalidade: *Print Working Directory*. Mostra o caminho absoluto completo do diretório em que o usuário se encontra no momento.

Sintaxe básica: `pwd`

Resultado obtido: Foi impresso na tela o caminho absoluto e completo do diretório em que o terminal estava operando no momento (por exemplo, `/home/valdir`).

Figura 8. Captura de tela do comando (`pwd`)



```
valdir@valdir-VirtualBox:~/Documentos$ pwd
/home/valdir/Documentos
```

Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

2.8. Comando *mkdir*

Finalidade: Cria um ou mais diretórios (pastas) novos no sistema.

Sintaxe básica: `mkdir [nome_do_novo_diretorio]`

Resultado obtido: O comando foi executado sem exibir mensagens, retornando ao prompt. Ao listar o diretório novamente (com `ls -a`), foi possível confirmar que a nova pasta (*IFMG*) havia sido criada com sucesso.

Figura 9. Captura de tela do comando (*mkdir*)

```
valdir@valdir-VirtualBox:~/Documentos$ mkdir IFMG
valdir@valdir-VirtualBox:~/Documentos$ dir
IFMG
valdir@valdir-VirtualBox:~/Documentos$ ls -a
.  ..  IFMG
```

Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

2.9. Comando *rmdir*

Finalidade: Remove um diretório, mas apenas se ele estiver completamente vazio.

Sintaxe básica: `rmdir [nome_do_diretorio]`

Resultado obtido: O diretório vazio especificado (*IFMG*) foi removido de forma silenciosa, retornando imediatamente ao prompt de comando.

Figura 10. Captura de tela do comando (*rmdir*)

```
valdir@valdir-VirtualBox:~/Documentos$ rmdir IFMG
valdir@valdir-VirtualBox:~/Documentos$ dir
valdir@valdir-VirtualBox:~/Documentos$ ls -a
.  ..
```

Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

2.10. Comando *touch*

Finalidade: Cria um arquivo vazio. Também é usado para atualizar a data e hora de modificação de um arquivo que já existe.

Sintaxe básica: `touch [nome_do_arquivo.extensao]`

Resultado obtido: O comando retornou ao prompt sem apresentar mensagens. A ação gerou com sucesso um arquivo de texto (*teste.txt*) completamente vazio no local especificado.

Figura 11. Captura de tela do comando (touch)

```
valdir@valdir-VirtualBox: ~/Documentos/IFMG$ touch teste.txt
valdir@valdir-VirtualBox: ~/Documentos/IFMG$ ls -a
.  ..  teste.txt
```

Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

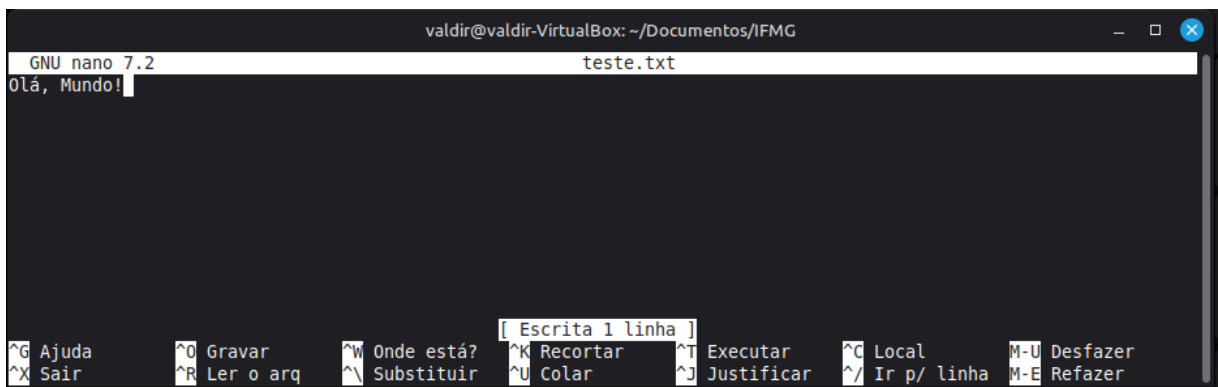
2.11. Comando *nano*

Finalidade: Abre o editor de texto nano diretamente no terminal, permitindo criar ou editar arquivos de texto de forma simples.

Sintaxe básica: nano [nome_do_arquivo]

Resultado obtido: A interface do terminal foi substituída pela tela do editor de texto GNU nano, permitindo a digitação do conteúdo. Após salvar o arquivo (*teste.txt*) e sair (Ctrl+X), a tela retornou ao prompt normal.

Figura 12. Captura de tela do comando (nano)



Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

2.12. Comando *cp*

Finalidade: Copia arquivos ou diretórios de um local para outro.

Sintaxe básica: cp [arquivo_origem] [diretorio_destino]

Resultado obtido: O arquivo (*teste.txt*) foi copiado para o diretório de destino de maneira silenciosa. O comando retornou ao prompt sem exigir confirmações, a menos que houvesse conflito de arquivos.

Figura 13. Captura de tela do comando (cp)

```
valdir@valdir-VirtualBox:~/Documentos/IFMG$ cp teste.txt /home/valdir/Documentos
valdir@valdir-VirtualBox:~/Documentos/IFMG$ cd ..
valdir@valdir-VirtualBox:~/Documentos$ ls -a
.  ..  IFMG  teste.txt
```

Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

2.13. Comando *mv*

Finalidade: Move arquivos/diretórios de um local para outro. Também é o comando utilizado para renomear arquivos.

Sintaxe básica: mv [origem] [destino]

Resultado obtido: O arquivo (*IFMG*) foi movido com sucesso, e o comando operou de forma invisível, liberando o terminal logo em seguida.

Figura 14. Captura de tela do comando (mv)

```
valdir@valdir-VirtualBox:~/Documentos$ mv IFMG /home/valdir/Downloads
valdir@valdir-VirtualBox:~/Documentos$ ls -a
.  ..  teste.txt
valdir@valdir-VirtualBox:~/Documentos$ cd ..
valdir@valdir-VirtualBox:~$ cd Downloads
valdir@valdir-VirtualBox:~/Downloads$ ls -a
.  ..  IFMG
```

Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

2.14. Comando *rm*

Finalidade: Remove (apaga) arquivos. (Para apagar diretórios com conteúdo, costuma-se usar rm -r).

Sintaxe básica: rm [nome_do_arquivo]

Resultado obtido: O arquivo especificado (*IFMG*) foi excluído permanentemente do diretório. O terminal processou a exclusão e retornou ao prompt sem exibir avisos ou confirmações adicionais.

Figura 15. Captura de tela do comando (rm)

```
valdir@valdir-VirtualBox:~/Documentos$ ls -a
.  ..  IFMG  teste.txt
valdir@valdir-VirtualBox:~/Documentos$ rm -r IFMG
valdir@valdir-VirtualBox:~/Documentos$ ls -a
.  ..  teste.txt
```

Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

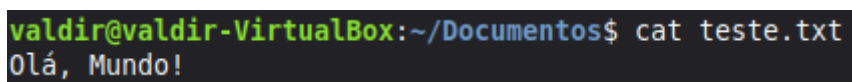
2.15. Comando *cat*

Finalidade: Concatena e exibe o conteúdo completo de um arquivo de texto diretamente na tela do terminal.

Sintaxe básica: `cat [nome_do_arquivo]`

Resultado obtido: O texto contido dentro do arquivo especificado (*teste.txt*) foi impresso na íntegra diretamente na tela do terminal, permitindo a sua leitura rápida.

Figura 16. Captura de tela do comando (*cat*)



```
valdir@valdir-VirtualBox:~/Documentos$ cat teste.txt
Olá, Mundo!
```

Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

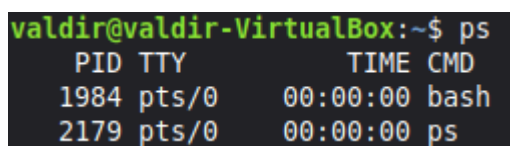
2.16. Comando *ps*

Finalidade: Mostra um instantâneo (*snapshot*) dos processos que estão rodando no sistema na sua sessão atual.

Sintaxe básica: `ps`

Resultado obtido: O terminal exibiu uma pequena tabela contendo informações dos processos rodando na sessão atual, incluindo o número identificador (PID), o terminal (TTY), o tempo de uso da CPU e o nome do comando.

Figura 17. Captura de tela do comando (*ps*)



```
valdir@valdir-VirtualBox:~$ ps
  PID TTY          TIME CMD
 1984 pts/0        00:00:00 bash
 2179 pts/0        00:00:00 ps
```

Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

2.17. Comando *top*

Finalidade: Exibe um gerenciador de tarefas em tempo real, mostrando os processos em execução e o uso de CPU e memória RAM.

Sintaxe básica: `top` (Pressione a tecla *q* para sair).

Resultado obtido: A tela foi substituída por um gerenciador dinâmico atualizado em tempo real, exibindo o uso geral de CPU e Memória RAM do sistema, seguido por uma lista dos processos que mais consumiam recursos.

Figura 18. Captura de tela do comando (top)

```
top - 10:51:53 up 59 min, 1 user, load average: 0,18, 0,12, 0,09
Tarefas: 238 total, 1 em exec., 237 dormindo, 0 parado, 0 zumbi
%CPU(s): 1,5 us, 0,6 sy, 0,0 ni, 97,7 id, 0,1 wa, 0,0 hi, 0,2 si, 0,0 st
MB mem : 5425,1 total, 3783,7 free, 1117,0 used, 787,4 buff/cache
MB swap: 2280,0 total, 2280,0 free, 0,0 used. 4308,1 avail mem
```

PID	USUARIO	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TEMPO+	COMANDO
1348	valdir	20	0	5545416	309672	170968	S	22,6	5,6	1:05.58	cinnamon
899	root	20	0	1301760	185072	110404	S	9,3	3,3	1:32.19	Xorg
1971	valdir	20	0	694996	46840	36180	S	1,3	0,8	0:05.73	gnome-terminal-
676	root	20	0	258556	14448	12476	S	0,7	0,3	0:01.84	touchegg
1194	valdir	20	0	236044	8236	7404	S	0,3	0,1	0:00.27	at-spi2-registr
1343	valdir	20	0	544172	42364	26852	S	0,3	0,8	0:00.48	cinnamon-launch
1773	valdir	20	0	449776	25148	19604	S	0,3	0,5	0:00.17	xdg-desktop-por
2181	valdir	20	0	14748	6120	3828	R	0,3	0,1	0:00.04	top
1	root	20	0	22392	13592	9544	S	0,0	0,2	0:01.60	systemd
2	root	20	0	0	0	0	S	0,0	0,0	0:00.03	kthreadd
3	root	20	0	0	0	0	S	0,0	0,0	0:00.00	pool_workqueue_release
4	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-rcu_gp
5	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-sync_wq
6	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-kvfree_rcu_reclaim
7	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-slub_flushwq
8	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-netns
11	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/0:0H-events_highpri
13	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-mm_percpu_wq
14	root	20	0	0	0	0	I	0,0	0,0	0:00.00	rcu_tasks_kthread
15	root	20	0	0	0	0	I	0,0	0,0	0:00.00	rcu_tasks_rude_kthread
16	root	20	0	0	0	0	I	0,0	0,0	0:00.00	rcu_tasks_trace_kthread
17	root	20	0	0	0	0	S	0,0	0,0	0:00.02	ksoftirqd/0
18	root	20	0	0	0	0	I	0,0	0,0	0:00.90	rcu_preempt
19	root	20	0	0	0	0	S	0,0	0,0	0:00.00	rcu_exp_par_gp_kthread_worker/0
20	root	20	0	0	0	0	S	0,0	0,0	0:00.25	rcu_exp_gp_kthread_worker
21	root	rt	0	0	0	0	S	0,0	0,0	0:00.10	migration/0
22	root	-51	0	0	0	0	S	0,0	0,0	0:00.00	idle_inject/0
23	root	20	0	0	0	0	S	0,0	0,0	0:00.00	cpuhp/0
24	root	20	0	0	0	0	S	0,0	0,0	0:00.00	cpuhp/1
25	root	-51	0	0	0	0	S	0,0	0,0	0:00.00	idle_inject/1
26	root	rt	0	0	0	0	S	0,0	0,0	0:00.27	migration/1
27	root	20	0	0	0	0	S	0,0	0,0	0:00.02	ksoftirqd/1
28	root	20	0	0	0	0	I	0,0	0,0	0:00.07	kworker/1:0-cgwb_release
29	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/1:0H-events_highpri
30	root	20	0	0	0	0	S	0,0	0,0	0:00.00	cpuhp/2
31	root	-51	0	0	0	0	S	0,0	0,0	0:00.00	idle_inject/2
32	root	rt	0	0	0	0	S	0,0	0,0	0:00.25	migration/2
33	root	20	0	0	0	0	S	0,0	0,0	0:00.19	ksoftirqd/2
35	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/2:0H-events_highpri
36	root	20	0	0	0	0	S	0,0	0,0	0:00.00	cpuhp/3
37	root	-51	0	0	0	0	S	0,0	0,0	0:00.00	idle_inject/3
38	root	rt	0	0	0	0	S	0,0	0,0	0:00.27	migration/3
39	root	20	0	0	0	0	S	0,0	0,0	0:00.25	ksoftirqd/3
41	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/3:0H-kblockd
42	root	20	0	0	0	0	S	0,0	0,0	0:00.00	cpuhp/4

Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

2.18. Comando *htop*

Finalidade: Uma versão melhorada, colorida e interativa do comando top (*Pode ser necessário instalá-lo antes rodando: sudo apt install htop*).

Sintaxe básica: htop (Pressione a tecla q para sair).

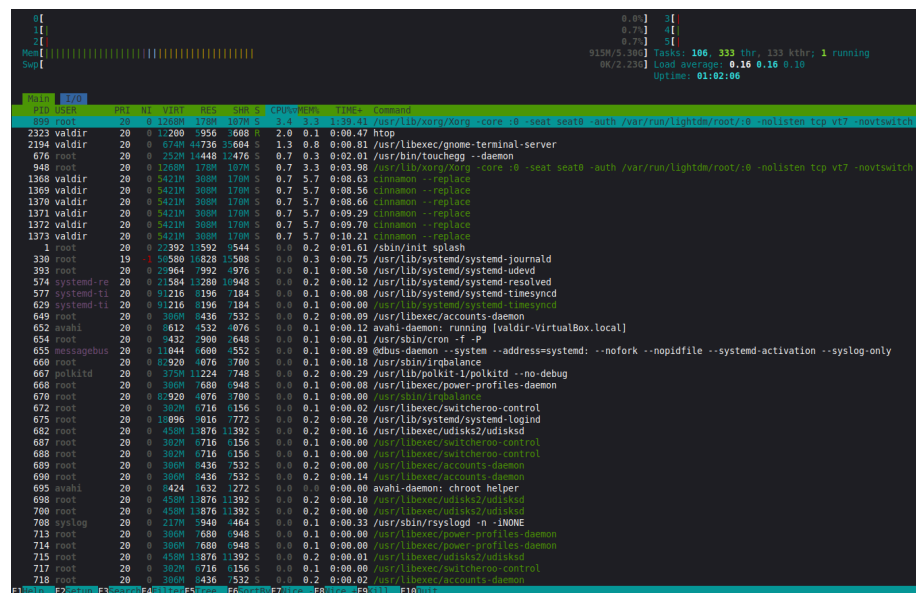
Resultado obtido: Foi aberta uma interface visual interativa e colorida, que apresentou gráficos em barras para o consumo de cada núcleo do processador e da memória RAM, facilitando o monitoramento do sistema operacional.

Figura 19. Captura de tela do comando (htop)

```
valdir@valdir-VirtualBox:~$ htop
Comando 'htop' não encontrado, mas poder ser instalado com:
sudo apt install htop
valdir@valdir-VirtualBox:~$ sudo apt install htop
[sudo] senha para valdir:
Lendo listas de pacotes... Pronto
Construindo árvore de dependências... Pronto
Lendo informação de estado... Pronto
Os NOVOS pacotes a seguir serão instalados:
  htop
0 pacotes atualizados, 1 pacotes novos instalados, 0 a serem removidos e 294 não atualizados.
É preciso baixar 171 kB de arquivos.
Depois desta operação, 434 kB adicionais de espaço em disco serão usados.
Obter:1 http://archive.ubuntu.com/ubuntu noble/main amd64 htop amd64 3.3.0-4build1 [171 kB]
Baixados 171 kB em 0s (847 kB/s)
A seleccionar pacote anteriormente não seleccionado htop.
(Lendo banco de dados ... 467466 ficheiros e diretórios atualmente instalados.)
A preparar para descompactar .../htop 3.3.0-4build1_amd64.deb ...
A descompactar htop (3.3.0-4build1) ...
Configurando htop (3.3.0-4build1) ...
A processar 'triggers' para mailcap (3.70+nmulubuntu1) ...
A processar 'triggers' para desktop-file-utils (0.27-2build1) ...
A processar 'triggers' para hicolor-icon-theme (0.17-2) ...
A processar 'triggers' para gnome-menus (3.36.0-1ubuntu3) ...
A processar 'triggers' para mate-menus (1.26.1+mint1) ...
A processar 'triggers' para man-db (2.12.0-4build2) ...
valdir@valdir-VirtualBox:~$ htop
valdir@valdir-VirtualBox:~$
```

Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

Figura 20. Captura de tela do comando (htop)



Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

2.19. Comando *kill*

Finalidade: Envia um sinal de término para forçar a parada de um processo específico com base no seu número identificador (PID).

Sintaxe básica: kill [PID_do_processo]

Resultado obtido: O processo correspondente ao PID informado (2397) recebeu o sinal de encerramento e foi finalizado. O comando retornou ao prompt de forma silenciosa.

Figura 21. Captura de tela do comando (kill)

PID	USUARIO	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TEMPO+	COMANDO
2397	valdir	20	0	4196036	522436	171808	R	114,7	9,4	0:21.95	firefox-bin
899	root	20	0	1346440	214096	128884	S	4,7	3,9	2:10.11	Xorg
1348	valdir	20	0	5545344	310228	175416	S	3,3	5,6	1:47.46	cinnamon
277	root	20	0	0	0	0	S	2,3	0,0	0:00.81	jbd2/sda3-8
101	root	0	-20	0	0	0	I	1,7	0,0	0:00.22	kworker/4:1H-kblockd
2374	valdir	20	0	691872	45056	35532	S	1,0	0,8	0:00.90	gnome-terminal-
2527	valdir	20	0	3545440	107780	72320	S	1,0	1,9	0:01.10	WebExtensions
85	root	20	0	0	0	0	I	0,7	0,0	0:04.49	kworker/2:1-events
1709	valdir	20	0	496920	63536	36468	S	0,7	1,1	0:07.86	mintreport-tray
164	root	0	-20	0	0	0	I	0,3	0,0	0:00.11	kworker/1:1H-kblockd
457	root	20	0	0	0	0	I	0,3	0,0	0:00.05	kworker/u28:2-ext4-rsv-conversion
585	root	20	0	0	0	0	I	0,3	0,0	0:00.07	kworker/u26:2-writeback
676	root	20	0	258556	14448	12476	S	0,3	0,3	0:02.66	touchegg
1647	valdir	20	0	831172	100164	55108	S	0,3	1,8	0:00.89	mintUpdate
2578	valdir	20	0	3570280	140668	93056	S	0,3	2,5	0:01.40	Privileged Cont
2698	valdir	20	0	3495760	71012	55676	S	0,3	1,3	0:00.29	Web Content
1	root	20	0	22392	13592	9544	S	0,0	0,2	0:01.64	systemd
2	root	20	0	0	0	0	S	0,0	0,0	0:00.03	kthreadd
3	root	20	0	0	0	0	S	0,0	0,0	0:00.00	pool_workqueue_release
4	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-rcu_gp
5	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-sync_wq
6	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-kvfree_rcu_reclaim
7	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-slub_flushwq
8	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-netns
11	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/0:0H-events_highpri
13	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-mm_percpu_wq
14	root	20	0	0	0	0	I	0,0	0,0	0:00.00	rcu_tasks_kthread
15	root	20	0	0	0	0	I	0,0	0,0	0:00.00	rcu_tasks_rude_kthread
16	root	20	0	0	0	0	I	0,0	0,0	0:00.00	rcu_tasks_trace_kthread
17	root	20	0	0	0	0	S	0,0	0,0	0:00.03	ksoftirqd/0
18	root	20	0	0	0	0	I	0,0	0,0	0:01.12	rcu_preempt
19	root	20	0	0	0	0	S	0,0	0,0	0:00.00	rcu_exp_par_gp_kthread_worker/0
20	root	20	0	0	0	0	S	0,0	0,0	0:00.25	rcu_exp_gp_kthread_worker
21	root	rt	0	0	0	0	S	0,0	0,0	0:00.11	migration/0
22	root	-51	0	0	0	0	S	0,0	0,0	0:00.00	idle_inject/0
23	root	20	0	0	0	0	S	0,0	0,0	0:00.00	cpuhp/0
24	root	20	0	0	0	0	S	0,0	0,0	0:00.00	cpuhp/1
25	root	-51	0	0	0	0	S	0,0	0,0	0:00.00	idle_inject/1
26	root	rt	0	0	0	0	S	0,0	0,0	0:00.28	migration/1
27	root	20	0	0	0	0	S	0,0	0,0	0:00.05	ksoftirqd/1
28	root	20	0	0	0	0	I	0,0	0,0	0:00.07	kworker/1:0-cgwb_release
29	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/1:0H-events_highpri
30	root	20	0	0	0	0	S	0,0	0,0	0:00.00	cpuhp/2
31	root	-51	0	0	0	0	S	0,0	0,0	0:00.00	idle_inject/2
32	root	rt	0	0	0	0	S	0,0	0,0	0:00.26	migration/2

```
valdir@valdir-VirtualBox:~$ kill 2397
valdir@valdir-VirtualBox:~$
```

Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

2.20. Comando *killall*

Finalidade: Encerra todos os processos que correspondem ao nome de um programa específico.

Sintaxe básica: killall [nome_do_programa]

Resultado obtido: Todos os processos em execução atrelados ao nome do programa especificado (*firefox-bin*) foram forçados a encerrar simultaneamente, sem exibir mensagens de retorno na tela.

Figura 22. Captura de tela do comando (killall)

PID	USUARIO	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TEMPO+	COMANDO
1348	valdir	20	0	5545344	309996	175248	S	4,7	5,6	1:58.60	cinnamon
899	root	20	0	1350776	218272	128724	S	3,0	3,9	2:20.18	Xorg
85	root	20	0	0	0	0	I	0,3	0,0	0:04.67	kworker/2:1-events
2830	valdir	20	0	4072636	321812	161008	S	0,3	5,8	0:06.01	firefox-bin
3202	valdir	20	0	14748	6148	3856	R	0,3	0,1	0:00.02	top
1	root	20	0	22392	13592	9544	S	0,0	0,2	0:01.64	systemd
2	root	20	0	0	0	0	S	0,0	0,0	0:00.03	kthreadd
3	root	20	0	0	0	0	S	0,0	0,0	0:00.00	pool workqueue_release
4	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-rcu_gp
5	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-sync_wq
6	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-kvfree_rcu_reclaim
7	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-slub_flushwq
8	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-netns
11	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/0:0H-events_highpri
13	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-mm_percpu_wq
14	root	20	0	0	0	0	I	0,0	0,0	0:00.00	rcu_tasks_kthread
15	root	20	0	0	0	0	I	0,0	0,0	0:00.00	rcu_tasks_rude_kthread
16	root	20	0	0	0	0	I	0,0	0,0	0:00.00	rcu_tasks_trace_kthread
17	root	20	0	0	0	0	S	0,0	0,0	0:00.03	ksoftirqd/0
18	root	20	0	0	0	0	I	0,0	0,0	0:01.19	rcu_preempt
19	root	20	0	0	0	0	S	0,0	0,0	0:00.00	rcu_exp_par_gp_kthread_worker/0
20	root	20	0	0	0	0	S	0,0	0,0	0:00.27	rcu_exp_gp_kthread_worker
21	root	rt	0	0	0	0	S	0,0	0,0	0:00.12	migration/0
22	root	-51	0	0	0	0	S	0,0	0,0	0:00.00	idle_inject/0
23	root	20	0	0	0	0	S	0,0	0,0	0:00.00	cpuhp/0
24	root	20	0	0	0	0	S	0,0	0,0	0:00.00	cpuhp/1
25	root	-51	0	0	0	0	S	0,0	0,0	0:00.00	idle_inject/1
26	root	rt	0	0	0	0	S	0,0	0,0	0:00.28	migration/1
27	root	20	0	0	0	0	S	0,0	0,0	0:00.08	ksoftirqd/1
28	root	20	0	0	0	0	I	0,0	0,0	0:00.07	kworker/1:0-events
29	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/1:0H-events_highpri
30	root	20	0	0	0	0	S	0,0	0,0	0:00.00	cpuhp/2
31	root	-51	0	0	0	0	S	0,0	0,0	0:00.00	idle_inject/2
32	root	rt	0	0	0	0	S	0,0	0,0	0:00.26	migration/2
33	root	20	0	0	0	0	S	0,0	0,0	0:00.27	ksoftirqd/2
35	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/2:0H-events_highpri
36	root	20	0	0	0	0	S	0,0	0,0	0:00.00	cpuhp/3
37	root	-51	0	0	0	0	S	0,0	0,0	0:00.00	idle_inject/3
38	root	rt	0	0	0	0	S	0,0	0,0	0:00.29	migration/3
39	root	20	0	0	0	0	S	0,0	0,0	0:00.27	ksoftirqd/3
41	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/3:0H-kblockd
42	root	20	0	0	0	0	S	0,0	0,0	0:00.00	cpuhp/4
43	root	-51	0	0	0	0	S	0,0	0,0	0:00.00	idle_inject/4
44	root	rt	0	0	0	0	S	0,0	0,0	0:00.29	migration/4
45	root	20	0	0	0	0	S	0,0	0,0	0:00.03	ksoftirqd/4

```

valdir@valdir-VirtualBox:~$ killall firefox-bin
valdir@valdir-VirtualBox:~$

```

Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

2.21. Comando jobs

Finalidade: Lista os processos que o usuário iniciou no terminal atual e que estão rodando em segundo plano ou suspensos.

Sintaxe básica: jobs

Resultado obtido: Foi exibida uma lista numerada contendo o status de processos que haviam sido suspensos (Parado) ou que estavam em execução em segundo plano na sessão atual do terminal.

Figura 23. Captura de tela do comando (jobs)

```
valdir@valdir-VirtualBox:~$ sleep 3000
^Z
[1]+  Parado                  sleep 3000
valdir@valdir-VirtualBox:~$ jobs
[1]+  Parado                  sleep 3000
valdir@valdir-VirtualBox:~$
```

Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

2.22. Comando *bg*

Finalidade: *Background*. Retoma a execução de um processo que foi suspenso, mas rodando-o em segundo plano para liberar o terminal.

Sintaxe básica: `bg` ou `bg %[numero_do_job]`

Resultado obtido: O terminal exibiu o nome do comando seguido do símbolo `&`, indicando que o processo que estava suspenso retomou sua execução, mas agora operando em segundo plano.

Figura 24. Captura de tela do comando (bg)

```
valdir@valdir-VirtualBox:~$ sleep 3000
^Z
[1]+  Parado                  sleep 3000
valdir@valdir-VirtualBox:~$ jobs
[1]+  Parado                  sleep 3000
valdir@valdir-VirtualBox:~$ bg 1
[1]+ sleep 3000 &
valdir@valdir-VirtualBox:~$
```

Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

2.23. Comando *fg*

Finalidade: *Foreground*. Traz um processo que estava rodando em segundo plano ou suspenso para o primeiro plano do terminal.

Sintaxe básica: `fg` ou `fg %[numero_do_job]`

Resultado obtido: O processo selecionado foi movido para o primeiro plano, assumindo o controle da tela do terminal e bloqueando o prompt até que a tarefa fosse concluída ou suspensa novamente.

Figura 25. Captura de tela do comando (fg)

```
valdir@valdir-VirtualBox:~$ sleep 3000
^Z
[1]+  Parado                  sleep 3000
valdir@valdir-VirtualBox:~$ jobs
[1]+  Parado                  sleep 3000
valdir@valdir-VirtualBox:~$ bg 1
[1]+ sleep 3000 &
valdir@valdir-VirtualBox:~$ fg 1
sleep 3000
```

Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

2.24. Comando *uname -a*

Finalidade: Exibe todas as informações sobre o sistema (nome do kernel, versão, nome da máquina, arquitetura do processador).

Sintaxe básica: `uname -a`

Resultado obtido: Foi impressa uma linha extensa contendo os dados gerais do sistema, como o nome do SO (Linux), nome da máquina (host), versão completa do kernel e a arquitetura do processador.

Figura 26. Captura de tela do comando (`uname -a`)

```
valdir@valdir-VirtualBox:~$ uname -a
Linux valdir-VirtualBox 6.14.0-37-generic #37~24.04.1-Ubuntu SMP PREEMPT_DYNAMIC
Thu Nov 20 10:25:38 UTC 2 x86_64 x86_64 x86_64 GNU/Linux
```

Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

2.25. Comando *uname -r*

Finalidade: Exibe especificamente a versão de lançamento (release) do kernel do Linux em uso.

Sintaxe básica: `uname -r`

Resultado obtido: O terminal exibiu de forma objetiva apenas a numeração correspondente à versão exata do kernel Linux em uso no momento da execução.

Figura 27. Captura de tela do comando (`uname -r`)

```
valdir@valdir-VirtualBox:~$ uname -r
6.14.0-37-generic
```

Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

2.26. Comando *hostname*

Finalidade: Mostra ou define o nome da sua máquina (host) na rede.

Sintaxe básica: `hostname`

Resultado obtido: O comando imprimiu na tela o nome definido para a máquina na rede (*valdir-VirtualBox*), retornando ao prompt em seguida.

Figura 28. Captura de tela do comando (`hostname`)

```
valdir@valdir-VirtualBox:~$ hostname  
valdir-VirtualBox
```

Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

2.27. Comando *df -h*

Finalidade: Mostra o espaço livre e ocupado no disco rígido do sistema. O parâmetro `-h` (human-readable) formata os tamanhos em Megabytes e Gigabytes para facilitar a leitura.

Sintaxe básica: `df -h`

Resultado obtido: Foi apresentada uma tabela detalhada mostrando as partições do disco rígido, o tamanho total, o espaço utilizado e o disponível. O uso do parâmetro `-h` formatou os valores para MB e GB, facilitando a leitura.

Figura 29. Captura de tela do comando (`df -h`)

```
valdir@valdir-VirtualBox:~$ df -h  
Sist. Arq.      Tam. Usado Disp.  Uso% Montado em  
tmpfs           543M  1,2M  542M    1% /run  
/dev/sda3       24G   9,8G   13G   43% /  
tmpfs           2,7G    0   2,7G    0% /dev/shm  
tmpfs           5,0M   8,0K   5,0M    1% /run/lock  
/dev/sda2       512M   6,2M  506M    2% /boot/efi  
tmpfs           543M  232K   543M    1% /run/user/1000
```

Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

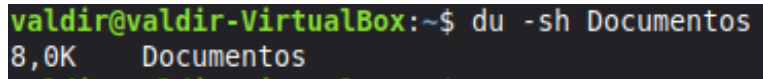
2.28. Comando *du -sh*

Finalidade: Calcula o tamanho total usado por um arquivo ou diretório específico. O `-s` resume o total e o `-h` formata para facilitar a leitura.

Sintaxe básica: `du -sh [caminho_do_diretorio_ou_arquivo]`

Resultado obtido: O comando calculou e exibiu na tela apenas o tamanho total do diretório (8 KB), utilizando uma formatação compreensível (como MegaBytes, GigaBytes ou KiloBytes como no meu caso).

Figura 30. Captura de tela do comando (du -sh)



```
valdir@valdir-VirtualBox:~$ du -sh Documentos
8,0K  Documentos
```

Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

2.29. Comando *lscpu*

Finalidade: Exibe informações detalhadas sobre a arquitetura da sua CPU (processador), como número de núcleos, threads, fabricante, etc.

Sintaxe básica: `lscpu`

Resultado obtido: A tela foi preenchida com um relatório técnico detalhado sobre a CPU da máquina, informando o modelo do processador, a arquitetura, o número de núcleos (cores), threads e os tamanhos da memória cache.

Figura 31. Captura de tela do comando (lscpu)

```
valdir@valdir-VirtualBox:~$ lscpu
Arquitetura:          x86_64
Modo(s) operacional da CPU: 32-bit, 64-bit
Tamanhos de endereço: 46 bits physical, 48 bits virtual
Ordem dos bytes:      Little Endian
CPU(s):               6
Lista de CPU(s) on-line: 0-5
ID de fornecedor:      GenuineIntel
Nome do modelo:        12th Gen Intel(R) Core(TM) i7-12700
Família da CPU:        6
Modelo:                151
Thread(s) per núcleo:  1
Núcleo(s) por soquete: 6
Soquete(s):            1
Step:                  2
BogoMIPS:              4224,00
Opções:                fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pg
                        e mca cmov pat pse36 clflush mmx fxsr sse sse2 ht s
                        yscall nx rdtscp lm constant_tsc rep_good nopl xtop
                        ology nonstop_tsc cpuid tsc_known_freq pni pclmulqd
                        q ssse3 cx16 pcid sse4_1 sse4_2 movbe popcnt aes rd
                        rand hypervisor lahf_lm abm 3dnowprefetch ibrs_enha
                        nced fsgsbase bmi1 bmi2 invpcid rdseed clflushopt m
                        d_clear flush_lld arch_capabilities

Recursos de virtualização:
Fabricante do hipervisor: KVM
Tipo de virtualização:    completo
Caches (soma de todos):
L1d:                       288 KiB (6 instâncias)
L1i:                       192 KiB (6 instâncias)
L2:                         7,5 MiB (6 instâncias)
L3:                        150 MiB (6 instâncias)
NUMA:
Nó(s) de NUMA:             1
CPU(s) de nó NUMA:         0-5
Vulnerabilidades:
Gather data sampling:      Not affected
Ghostwrite:                Not affected
Indirect target selection: Mitigation; Aligned branch/return thunks
Itlb multihit:             Not affected
L1tf:                      Not affected
Mds:                       Not affected
Meltdown:                  Not affected
Mmio stale data:           Not affected
Reg file data sampling:    Mitigation; Clear Register File
Retbleed:                  Mitigation; Enhanced IBRS
Spec rstack overflow:      Not affected
Spec store bypass:         Vulnerable
Spectre v1:                Mitigation; usercopy/swaps barriers and __user poi
                        nter sanitization
Spectre v2:                Mitigation; Enhanced / Automatic IBRS; PBRSE-eIBRS
                        SW sequence: BHT SW loop KVM SW loop
```

Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

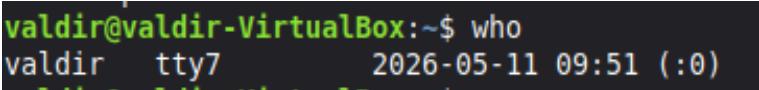
2.30. Comando *who*

Finalidade: Lista os usuários que estão conectados ao sistema no momento.

Sintaxe básica: `who`

Resultado obtido: O terminal exibiu uma lista com os nomes dos usuários logados no sistema naquele momento, incluindo a identificação do terminal que estavam utilizando e a data/hora do login.

Figura 32. Captura de tela do comando (who)



```
valdir@valdir-VirtualBox:~$ who
valdir  tty7          2026-05-11 09:51 (:0)
```

Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

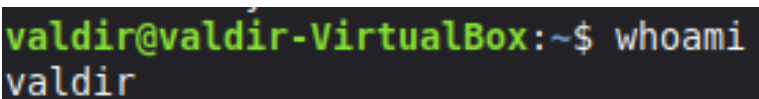
2.31. Comando *whoami*

Finalidade: Exibe apenas o nome de usuário com o qual o usuário está logado e operando no terminal naquele exato momento.

Sintaxe básica: whoami

Resultado obtido: O comando imprimiu no terminal exclusivamente o nome de usuário (*valdir*) sob a qual a sessão atual estava operando.

Figura 33. Captura de tela do comando (whoami)



```
valdir@valdir-VirtualBox:~$ whoami
valdir
```

Fonte: Captura de tela retirada pelo autor em 11 de maio de 2026.

3. CONSIDERAÇÕES FINAIS

3.1. Comandos mais úteis

Durante a prática, os comandos de navegação e visualização de diretórios, como `ls`, `cd` e `pwd`, mostraram-se os mais úteis para a rotina básica de manipulação do sistema, pois criam a base necessária para se localizar no ambiente via texto. Do ponto de vista técnico e analítico, os comandos `top`, `htop` e `ps` foram extremamente úteis, pois permitiram visualizar em tempo real o consumo de recursos (CPU e Memória) e entender como o sistema operacional distribui o processamento entre as diversas tarefas ativas.

3.2. Comandos com mais dificuldade

A maior complexidade de execução concentrou-se nos comandos de gerenciamento de tarefas em segundo plano (*jobs*, *bg* e *fg*). A dificuldade ocorre porque esses comandos exigem que o usuário trabalhe com processos que não estão visíveis na tela principal, demandando uma abstração maior sobre como o sistema suspende e retoma atividades. Além disso, o uso correto dos comandos *kill* e *killall* exige atenção redobrada, pois é preciso identificar com precisão o identificador do processo (PID) para não encerrar aplicações erradas ou vitais para o sistema.

3.3. Como o Shell se relaciona com os conceitos de Sistemas Operacionais estudados em sala.

A prática no Shell materializa toda a teoria arquitetural dos Sistemas Operacionais vista em sala de aula. A relação mais direta ocorre através das Chamadas de Sistema (System Calls) e dos Modos de Operação. Quando digitamos um comando no Shell (como *mkdir* para criar uma pasta), estamos no *Modo Usuário*. O Shell interpreta esse comando e dispara uma chamada de sistema (interrupção de software) solicitando que o Kernel assuma o controle no *Modo Privilegiado (Modo Núcleo)* para realizar a alteração física no disco rígido.

Além disso, a prática com o Shell tornou visível a teoria de Processos e Concorrência. Ao utilizar comandos como *ps* e *top*, foi possível observar o sistema operacional gerindo o Bloco de Controle de Processos (PCB) de dezenas de aplicações ao mesmo tempo, aplicando algoritmos de escalonamento e transitando os processos entre os estados de "Pronto", "Em Execução" e "Espera" de forma simultânea. O terminal é a janela que nos permite ver o Kernel trabalhando.